

Full Text of Goldshine Protocol Version 1.5.2: A Decentralized Global Capability Delivery Network Based on Agent Encapsulation

Full Text of Goldshine Protocol Version 1.5.2

A Decentralized Global Capability Delivery Network Based on Agent Encapsulation

Protocol Design, Formal Definition, and Simulation Verification

Goldshine Protocol: A Decentralized Global Capability Delivery Network Based on Agent Encapsulation

Authors: GCAT / DeepSeek

Abstract

The current agent ecosystem faces three core bottlenecks: the intractable problem of context pollution (logical isolation schemes cannot eliminate memory crosstalk), the coarse granularity of capability reuse (tool protocols such as MCP deliver functions rather than finished products), and the high cost of cross-entity collaboration (significant friction arising from heterogeneous environments, language barriers, and a lack of trust). To address these issues, this paper proposes the Goldshine Protocol—a decentralized capability delivery network protocol that upgrades the granularity of AI capability delivery from the "tool function level" to the "autonomous agent level."

The core contributions of the protocol include: (1) Proposing a formal five-tuple definition for the Agent Service Unit (ASU), which effectively addresses context pollution at the architectural level through physical-level memory isolation; (2) Designing the Goldshine Semantic Ontology and a dual-layer registration and

discovery mechanism to enable intent-driven, precise capability matching and fully language-agnostic interaction; (3) Constructing a decentralized trust system incorporating multi-dimensional reputation, staking slashing, and tiered arbitration, and verifying its convergence and attack resistance through a multi-agent evolutionary game theory simulation framework; (4) Proposing a full supply chain collaboration framework spanning the digital and physical worlds, connecting the entire delivery chain; (5) Implementing a minimum viable prototype (MVP) to validate the feasibility of the core paradigm from an engineering perspective.

This paper fully presents the protocol's four-layer technical architecture, core interaction specifications, economic governance mechanisms, and formal definition. It also designs two reproducible experimental frameworks—an information attenuation simulation and a reputation game theory simulation—and provides expected results based on theoretical derivation [SIMULATION]. The research demonstrates that this paradigm can solve the context pollution problem at the architectural level, significantly reduce the matching and communication friction in cross-entity collaboration, and offer a viable open protocol for distributed capability collaboration in the AI era.

Keywords: Goldshine Protocol; Agent Service Unit (ASU); Decentralized Network; Semantic Ontology; Interface Contract; Multi-dimensional Reputation; Formal Definition; Minimum Viable Prototype (MVP)

Chapter 1: Introduction

1.1 Research Background and Problem Statement

The rapid iteration of large language models (LLMs) has continuously enhanced the general capabilities of single models [4]. However, three core bottlenecks remain unresolved in engineering practice:

First, the persistent problem of context pollution. In multi-tenant scenarios, session memories from different users frequently interfere with each other under logical isolation schemes (e.g., prompt engineering, vector tagging). The industry currently lacks an architectural-level solution. Carefully crafted prompt injection attacks can breach logical isolation and access the context data of other users.

Second, the coarse granularity of capability reuse. The current mainstream approach to capability reuse remains at the "tool function level"—Function Calling exposes atomic function signatures [8], and the MCP protocol unifies the tool calling interface but does not encapsulate the complete task process [9]. The caller must still orchestrate tool sequences, manage context, and handle exceptions. This is essentially "delivering parts and an instruction manual" rather than "delivering a finished product."

Third, the high friction in cross-entity collaboration. Heterogeneous runtime environments, differences in technology stacks, language barriers, and a lack of trust

make it nearly impossible to achieve cross-organizational, cross-regional AI capability collaboration without a centralized platform. The cost of matching capability providers with demanders is high, and long-tail capabilities are difficult to discover.

1.2 An Illustrative Experiment

In early 2026, the author conducted the following experiment on personal infrastructure: A "long-form web novel writing agent" (equipped with hardened prompts, an MCP toolchain, a standardized workflow, and an independent memory system) that had been fine-tuned over a long period on one server was fully encapsulated as an HTTP API endpoint compatible with the OpenAI Chat Completions format. This endpoint was then called by a local agent with no writing skills whatsoever. The result showed that the local agent, with just a single user instruction, received a complete chapter as a finished deliverable.

This phenomenon reveals a paradigm shift not yet fully recognized by the industry: When an agent is encapsulated as a service rather than decomposed into tools, the granularity of AI capability delivery upgrades from the "function calling level" to the "task delivery level." This is the fundamental difference between "lending you a wrench so you have to fix the car yourself" and "giving the car to him, and he returns it fixed."

1.3 From a Single Node to a Network

This encapsulated agent has effectively become an autonomous capability node—possessing an independent reasoning engine, independent memory, an independent skill package, and a standardized external interface. It can provide services independently without any centralized scheduler. If there were thousands or millions of such nodes, each encapsulating specialized capabilities in different domains, could they form a network? Could a demander automatically discover and invoke the most suitable node? Could complex tasks be automatically decomposed and distributed to multiple nodes for collaborative completion?

Packaging a single node is an engineering skill. Interconnecting ten thousand nodes creates a network effect. When an open protocol allows anyone to freely package capabilities and connect to the network, this constitutes a restructuring of production relations.

1.4 Contributions of This Paper

This paper is the second part of the author team's "Agent Operating System (AgentOS)" research framework. The first part, "Agent OS: A Single-Node Agent Operating System (AgentOS) for Capability Encapsulation" [22], proposed a standardized architecture for single-node agents, addressing modular construction, unified runtime, and capability encapsulation. Building on this foundation, this paper extends the research scope from a single node to a global network, proposing interconnection protocols, collaboration mechanisms, and an economic governance system for agents, thereby forming a complete technology stack from a "single-node

operating system" to a "global value network."

The core contributions of this paper are as follows:

1. **Paradigm Innovation:** Proposes the "agent-level capability reuse" paradigm, upgrading the granularity of capability delivery from the function level to the autonomous agent level, effectively addressing the context pollution problem at the architectural level.
2. **Protocol Design:** Fully designs the four-layer technical architecture, core interaction specifications, and economic governance mechanisms of the Goldshine Protocol, including subsystems such as semantic ontology, two-tier discovery, tiered arbitration, and token anchoring.
3. **Formal Definition:** Provides rigorous mathematical definitions and boundary constraints for core concepts such as ASU, reputation calculation, and matching and ranking.
4. **Simulation Verification:** Designs two reproducible experimental frameworks—information attenuation simulation and reputation game theory simulation—providing a verification path for the protocol's effectiveness.
5. **Engineering Demonstration:** Implements a minimum viable prototype (MVP), verifying the feasibility of the core paradigm from an engineering perspective and providing a reproducible baseline implementation for the community.

1.5 Structure of This Paper

Chapter 2 reviews related work; Chapter 3 provides the formal definition and internal structure of the ASU; Chapter 4 elaborates on the semantic ontology, interface contract, and two-tier discovery mechanism; Chapter 5 discusses multi-node collaborative orchestration and information attenuation control; Chapter 6 describes the full supply chain integration in the physical world; Chapter 7 constructs the trust and economic system; Chapter 8 discusses open guarantees, compliance, and cold start; Chapter 9 presents scenario deduction and paradigm verification; Chapter 10 conducts comparative analysis and feasibility demonstration; Chapter 11 discusses challenges and future work; Chapter 12 provides the conclusion; the appendix provides a glossary and a complete reproduction guide for the minimum viable prototype (MVP).

Chapter 2 Related Work

2.1 Agent Engineering: From Tool Calling to Memory Engineering

The development of agent engineering proceeds along two main lines. In tool calling,

from OpenAI Function Calling [8] to the Anthropic MCP protocol [9], the core idea is to abstract external capabilities into standardized function interfaces. The common limitation of these approaches is that they deliver "parts," leaving the assembly work to the caller.

In memory engineering, techniques such as RAG, vector databases, and session state management focus on maintaining context within a single agent [10]. However, these solutions all employ logical isolation and cannot fundamentally prevent memory interference in multi-tenant scenarios—this is the core problem that this protocol aims to solve through physical isolation.

2.2 Multi-Agent Systems: From Centralized Scheduling to Closed Environments

Traditional MAS (Multi-Agent System) theory has laid a formal foundation for multi-agent collaboration [11]. In engineering practice, frameworks such as AutoGen [12] and ChatDev [13] have implemented task orchestration for multiple agents, but their common feature is the need for a centralized scheduler operating in a closed, homogeneous environment, which does not support cross-organizational, cross-entity market-based collaboration. The Goldshine Protocol extends the collaboration boundary from a "closed system of a single organization" to a "globally open decentralized network."

2.3 Decentralized Service Networks: From Web Services to Computing Power Networks

The RESTful architecture [2] has achieved standardized invocation of web services, but service discovery still relies on a centralized registry. Blockchain networks [5][6] have achieved decentralized value transfer and smart contracts, but they are primarily oriented toward financial scenarios.

The field of Semantic Web Services conducted extensive early exploration into the standardized description and automatic matching of services [18][19], using ontology languages such as OWL-S to describe the inputs, outputs, and functions of web services. However, their limitations include coarse description granularity, high adaptation costs, a focus solely on information services, and a lack of supporting trust and transaction mechanisms. Building on this line of thought, the Goldshine Protocol simplifies ontology design by leveraging the semantic understanding capabilities of large language models (LLMs), upgrades the service granularity from interfaces to complete agents, and supplements the complete economic system for market-based collaboration.

The recently proposed Agent Network Protocol (ANP) establishes a meta-protocol foundation for decentralized agent networks [21]. By employing a DID identity system, a meta-protocol negotiation mechanism, and the ADP discovery protocol, it enables cross-platform interoperability among heterogeneous agents. This is the first protocol to systematically address the problem of decentralized agent interconnection, filling a

research gap in the dimension of openness.

However, ANP still has three core limitations: First, the granularity of capability remains at the function-calling level, defining only standard interfaces for tool calling without involving the encapsulation and delivery of complete tasks. Second, it does not cover the physical world, supporting only interactions with digital services and lacking specifications for physical manufacturing, logistics, and offline services. Third, it lacks a formal definition and an economic system, providing no rigorous mathematical definitions for agent units, trust mechanisms, or collaboration models, nor does it design an economic governance framework to support market-based collaboration.

Building upon ANP's decentralized interconnection, the Goldshine Protocol upgrades the granularity of capability delivery to the complete agent level, extends support to the full supply chain of the physical world, and supplements a complete formal definition, trust system, and economic mechanism. This forms a comprehensive solution ranging from single-node encapsulation to global network collaboration.

Decentralized computing networks (e.g., Golem, Render Network) tokenize computing resources, but they deliver "computing power" rather than "complete task outcomes." The Goldshine Protocol elevates the delivery granularity of decentralized networks from "computing resources" to "finished professional capabilities."

2.4 Summary of Research Gaps

Existing research exhibits gaps in the following three dimensions:

1. Granularity Dimension: The encapsulation and delivery of capabilities at the "agent level" (rather than the tool level or computing power level) has not been achieved.
2. Systemic Dimension: A complete solution covering both the digital and physical worlds, and encompassing both technical and economic systems, has not been constructed.
3. Rigor Dimension: Core concepts lack rigorous formal definitions, and there is no reproducible simulation verification framework.

The intersection of these three gaps defines the research focus of this paper.

Chapter 3: Formal Definition and Internal Architecture of the Agent Service Unit (ASU)

3.1 Formal Definition

Definition 1 (Agent Service Unit (ASU)): An Agent Service Unit (ASU) is a five-tuple: $ASU = (E, D, W, M, I)$

E (Base Inference Engine): Satisfies that for any input conforming to the protocol specification, it generates an output, i.e., $\cdot$ can be any large language model (LLM) or traditional program logic; the protocol does not prescribe a specific implementation. $x \in \mathcal{X}, y \in \mathcal{Y}: \mathcal{X} \rightarrow \mathcal{Y}$

D (Domain Enhancement Layer): Defined as a four-tuple $\cdot$. Here, $\cdot$ is the set of system prompts, defining the ASU's role, boundaries, and methodology; $\cdot$ is the few-shot example library; $\cdot$ is the domain knowledge graph; and $\cdot$ is the set of chain-of-thought templates, where each template is a standardized sequence of reasoning steps for a typical task in that domain. $D = (\mathcal{P}, \mathcal{F}, \mathcal{K}, \mathcal{C})$ $\mathcal{PF} = \{(x_i, y_i)\}_{i=1}^n \mathcal{KC}$

W (Workflow Engine): Defined as an extended finite state machine (FSM) with inputs and outputs: $W = (S, s_0, \Sigma, \Omega, \delta, F, E)$

$$Ss_0 \in S\Sigma\Omega\delta: S \times \Sigma \rightarrow S \times \Omega F \subseteq SE \subseteq SW$$

M (Memory System): Defined as $\cdot$. $\cdot$ is session memory (in-memory, not persisted); $\cdot$ is task memory (persisted, project-level state, isolated by client_namespace); and $\cdot$ is long-term knowledge memory (persisted, accumulated across tasks, independent knowledge base). $M = (M_{\text{session}}, M_{\text{task}}, M_{\text{long}})$ $M_{\text{session}} M_{\text{task}} M_{\text{long}} \forall c_1 \neq c_2, M_{\text{task}}(c_1) \cap M_{\text{task}}(c_2) = \emptyset$

I (Protocol Adaptation Layer): Responsible for bidirectional conversion between external standard interfaces and internal logic: $\cdot$. It also includes a built-in capability boundary validation function $\cdot$. If the input exceeds the registered capability scope, it returns the error code GSE_OUT_OF_SCOPE and refuses execution. $I: \mathcal{X}_{\text{protocol}} \leftrightarrow \mathcal{X}_{\text{internal}} \text{scope}(x)$

3.2 Physical Isolation Mechanism

- **Instance-level isolation:** Each ASU is deployed in an independent container or virtual machine, possessing its own memory space, file system, and persistent volume. There are no shared memory channels between different ASUs.
- **Namespace isolation:** Each caller is assigned a globally unique `client\_namespace`. The ASU's memory management layer enforces that all read and write operations carry this identifier at the underlying level, ensuring that data across different callers is physically inaccessible to each other.

Comparison with existing solutions: Current mainstream LLM applications employ logical isolation based on prompt words or vector labels [10], which cannot prevent context leakage caused by carefully crafted prompt injection attacks. This protocol, through physical-level isolation, reduces the risk of context pollution to the difficulty level of container escape attacks. According to industry security data, the average success probability of container escape attacks is approximately 10^{-6} , while the success probability of prompt injection attacks under logical isolation is approximately 10^{-2} . That is, physical isolation reduces the risk of context leakage by about 10,000 times, which in engineering practice can be considered an effective solution to the context pollution problem in conventional scenarios. $10^{-6} 10^{-2}$

3.3 Deployment Cost Tiers

For ASU providers of different scales, the protocol supports three deployment modes, corresponding to different resource consumption and costs:

Level	Deployment Mode	Applicable Scenario	Cost Characteristics
L1 Independent Deployment	Dedicated Instance	High-Frequency Core ASU	Dedicated resources, high cost, optimal performance
L2 Shared Foundation	Shared inference engine, independent memory	Long-tail Low-Frequency ASU	Cost reduced to 20–30% of L1
L3 On-Demand Wake-up	Dormant resident, triggered on task initiation	Extremely Low-Frequency ASU	Idle cost approaches zero

Providers can dynamically switch between the three modes based on actual order volume and revenue. This aligns with the natural pattern of market growth—when business is small, shared solutions are used to reduce costs; as business grows, it naturally upgrades to independent deployment to handle more orders. The protocol layer does not impose mandatory intervention.

3.4 Capability Boundary Self-Check

Definition 2 (Capability Boundary Check) For an ASU and its registered capability label set , the task resolves to a required label set . The capability boundary check function is:

$$\begin{aligned}
 & a\mathcal{L}_{\text{reg}}(a)x\mathcal{L}_{\text{demand}}(x)\text{scope}(a, x) = \\
 & \begin{cases} \text{true} & \text{if } \mathcal{L}_{\text{demand}}(x) \subseteq_{\text{semantic}} \mathcal{L}_{\text{reg}}(a) \\ \text{false} & \text{otherwise} \end{cases}
 \end{aligned}$$

$\subseteq_{\text{semantic}}$

Chapter 4: Semantic Ontology, Interface Contract, and Two-Layer Discovery

4.1 Goldshine Semantic Ontology Specification

The semantic ontology is the common language for describing the capabilities of all ASUs across the network and matching requirements. It is the foundational infrastructure for language-agnostic interaction.

Definition 3 (Goldshine Semantic Ontology) The Goldshine Semantic Ontology is defined as a three-level hierarchical classification system: $\mathcal{OO} = (\mathcal{D}_{\text{domain}}, \mathcal{F}, \mathcal{L}, \mathcal{H}, \mathcal{R})$

$\mathcal{D}_{\text{domain}}$ is the set of Domains, with the current version containing 16 first-level domains;

$\mathcal{F}$ is the set of Functions, with each domain containing several functions;

$\mathcal{L}$ is the set of free-form Labels, allowing providers to define fine-grained custom labels;

$\mathcal{H} \subseteq (\mathcal{D}_{\text{domain}} \times \mathcal{F}) \cup (\mathcal{F} \times \mathcal{L})$ is the hierarchical relationship, defining parent-child affiliations;

$\mathcal{R}$ is the set of mapping rules, used for semantic mapping from custom labels to standard labels.

Definition 4 (Semantic Containment) Given an ontology hierarchy structure , for a required label set and a registered label set , it is said that is semantically contained in , denoted as , if and only if: $\mathcal{H} L_d L_r L_d L_r L_d \subseteq_{\text{semantic}} L_r$

1. For any , there exists such that is a descendant node of in the ontology hierarchy (i.e., a more fine-grained subcategory); $l_d \in L_d L_r \in L_r l_d L_r$
2. For labels that cannot be matched hierarchically, their semantic similarity is , with a default threshold of . $\text{Sim}_{\text{vec}}(l_d, l_r) \geq \theta_s \theta_s = 0.85$

Semantic ambiguity resolution: When the same task can be classified into multiple domains, the intent parsing engine outputs a domain probability distribution , taking as the primary domain. Other domains with a probability above the threshold are used as auxiliary labels for matching. $P(d|x) \arg \max P(d|x) \theta = 0.3$

Semantic matching algorithm: The matching degree between user intent and ASU capability description adopts a fusion algorithm: $ua\text{Sim}(u, a) = \lambda \cdot \text{Sim}_{\text{tag}}(u, a) + (1 - \lambda) \cdot \text{Sim}_{\text{vec}}(\mathbf{v}_u, \mathbf{v}_a)$

$$\text{Sim}_{\text{tag}} \text{Sim}_{\text{vec}} \lambda = 0.4$$

The technical connotation of language-agnostic: The core of language-agnostic is to decouple human natural language from the interaction protocol between nodes:

1. Semantic frame uses a standardized symbolic encoding system (not any human natural language). Labels and fields are globally unique machine-readable identifiers, unbound from natural languages such as English or Chinese;
2. Only at the rendering layer of the user terminal are symbolic identifiers translated

into the user's native language for presentation;

3. Within an ASU, prompts and knowledge bases in any language may be used, but protocol-layer interactions are entirely based on standardized symbols, achieving language decoupling between supply and demand.

4.2 Interface Contract

The core interaction interface of the Goldshine Protocol is fully compatible with the OpenAI Chat Completions API [8], enabling zero-cost ecosystem integration. Six optional extension fields are added to the native fields: `task_id` (global task identifier), `session_id` (session continuation), `capability_tags` (capability tags), `delivery_schema` (delivery format constraints), `callback_url` (asynchronous callback), and `max_budget` (budget ceiling).

Response specifications are divided into three categories:

- **Synchronous response:** Tasks that can be completed within a single HTTP request return a standard Chat Completion format, with the addition of the `goldshine_ext` write-back field.
- **Streaming response:** Compatible with the SSE standard [15], supporting real-time progress push.
- **Asynchronous response:** For long-duration tasks, returns 202 Accepted, providing `poll_url` and `estimated_completion`.

4.3 Dual-Layer Registration and Discovery

To address the synchronization issue between DHT static metadata and real-time status, a dual-layer architecture [7] is adopted:

- **L1 Static Layer (DHT):** Stores metadata that changes infrequently—capability descriptions (in semantic ontology format), interface addresses, public keys, qualification certificates, and historical reputation summaries. The update frequency is at the hour/day level.
- **L2 Dynamic Layer (Real-time probing):** After roughly filtering the Top-20 candidates from the DHT during scheduling, real-time probes are sent in parallel to the `/health` endpoints of these ASUs to obtain current load, real-time quotes, and online status.

Scheduling process: Intent parsing → Semantic frame generation → DHT static layer rough filtering to Top-20 → Real-time probing for dynamic status → Comprehensive ranking → Default automatic selection of the optimal candidate; advanced users can manually select.

4.4 Formal Definition of Scheduling Matching

Definition 5 (Matching Ranking Function) Given user intent and candidate ASU set, a pre-filtering step is first performed: only candidates satisfying and are

retained (is the quote, is the user budget; is the current load, is the maximum concurrency). Nodes exceeding the budget or at full load are directly filtered out and do not participate in ranking. $u \mathcal{A} = \{a_1, a_2, \dots, a_n\} P(a) \leq P_{max} L(a) \leq L_{max} P(a) P_{max} L(a) L_{max}$

For the filtered candidate set, the matching ranking score is: $\text{Score}(u, a) = w_1 \cdot$

$$\text{Sim}(u, a) + w_2 \cdot R(a) + w_3 \cdot \left(1 - \frac{P(a)}{P_{max}}\right) + w_4 \cdot \left(1 - \frac{L(a)}{L_{max}}\right)$$

$$\text{Sim}(u, a) R(a) (w_1, w_2, w_3, w_4) = (0.35, 0.30, 0.20, 0.15)$$

4.5 Metadata Anti-Counterfeiting and Byzantine Defense

Byzantine nodes in a Distributed Hash Table (DHT) network may publish false ASU metadata, contaminating scheduling results through Sybil attacks. Defense mechanisms:

1. **Signature endorsement:** All metadata is signed by the publisher's DID [10], and the scheduling node accepts it only after verification. False information directly deducts the publisher's reputation score.
2. **Minimum staking threshold:** ASUs must stake a minimum amount of GOLD tokens as collateral for malicious behavior when registering on the network. If false registration information is verified, all staked tokens are confiscated, and the associated subject DID is blacklisted, significantly increasing the cost of attacks.
3. **Cross-validation:** Multiple scheduling nodes sample and verify the same ASU, sharing results on-chain. Inconsistencies trigger a reduction in the node's weight.
4. **New node circuit breaker:** For the first 10 invocations of a newly registered ASU, the scheduling node automatically verifies delivery quality. Failure results in immediate registration rollback and confiscation of staked tokens.

4.6 Language-agnostic semantic layer

Natural language exists only at the outermost user interface layer. The scheduling layer converts user requests in any language into language-independent, structured semantic frames. The ASU interaction layer only processes semantic frames. The language of the requester and the language of the provider are completely decoupled—a cobbler ASU that only speaks Italian can be seamlessly invoked by a Chinese user.

Chapter 5 Multi-Node Collaboration: Task Orchestration and Information Attenuation Control

5.1 Formal definition of composite tasks

Definition 6 (Composite Task) A composite task is defined as a directed acyclic graph (DAG): $TT = (V, E, \phi, \psi)$

$$V = \{v_1, v_2, \dots, v_k\} E \subseteq V \times V (v_i, v_j) \in E \implies v_i \phi: V \rightarrow \mathcal{L} \psi: V \rightarrow \mathcal{O}$$

5.2 Dynamic DAG Adjustment

The orchestration engine supports dynamic adjustments during execution: automatically adding, deleting, or modifying downstream nodes based on intermediate results; allowing users to change requirements at manual confirmation points and perform local re-orchestration. Manual confirmation points are set at key nodes, where users can choose to continue, modify the direction, or terminate.

5.3 Collaborative Information Attenuation Model and Control

Problem analysis: Multi-layer ASU relay leads to information attenuation. Let the single-layer information retention rate be p , then the cumulative retention rate after k layers of invocation is $p^k \in (0,1)$.

All data in this chapter are simulated expected results; actual measured data will be supplemented in future work.

Experiment objectives: Quantify the information attenuation rate in multi-layer invocations and verify the effectiveness of the control scheme.

- Control group (CG): Pure relay chain without protection mechanisms
- Experiment group A (EG-A): Only Schema constraints added
- Experiment group B (EG-B): Only original requirement passthrough added
- Experiment group C (EG-C): Schema constraints + original requirement passthrough

Expected Results Framework:

Depth	CG Fidelity	CG Similarity	EG-C Fidelity	EG-C Similarity
1 Layer	4.2±0.5	0.91±0.04	4.4±0.3	0.94±0.02
2 Layers	3.5±0.7	0.78±0.08	4.1±0.4	0.89±0.04
3 Layers	2.8±0.9	0.61±0.12	3.7±0.5	0.83±0.06

Expected Conclusion: Under three-layer invocation, the combined scheme (EG-C) can increase information retention from approximately 60% to approximately 83%.

Control Scheme Summary:

1. **Delivery Schema Enforcement:** The output of each subtask must conform to a predefined JSON Schema, with automated validation of output field completeness and type correctness. Non-compliant outputs are automatically rejected and returned for rework.
2. **Automatic Intermediate Result Validation:** The orchestration engine performs semantic consistency checks on upstream outputs at key convergence nodes. If the result falls below a threshold, a clarification loop is automatically triggered.
3. **Transparent Transmission of Original Requirements:** Each subtask ASU simultaneously receives the global original requirement Abstract and the current subtask description, ensuring that the core intent is not lost.

5.4 Recursive Call Safety

Hard Depth Limit for Calls: The protocol specifies a maximum call depth between ASUs. Exceeding this limit returns the error code `GSE_DEPTH_EXCEEDED`. $d_{max} = 3$

Signature Chain Mechanism: The scheduling node issues an initial JWT token containing for the task, signed by the scheduling node's private key. $\{parent_task_id, call_depth, timestamp\}$

- **Centralized Scheduling Scenario:** For each layer of downward distribution, the scheduling node increments the depth and re-signs the token.
- **ASU Peer-to-Peer Call Scenario:** The caller must carry the original JWT token issued by the scheduling node, increment the `call_depth` field by 1 locally before passing it, and has no authority to issue a new token. The called ASU queries the scheduling node's public key via DIDRegistry[10] to verify the signature and checks whether the incremented depth is ≤ 3 .

An ASU cannot forge a signature on its own, ensuring the depth limit is cryptographically enforced. Call requests without the original scheduling token should be directly rejected by the ASU.

5.5 Equity Collaboration and Human-AI Hybrid Setup

Supports a collaboration model based on "contributing capabilities as equity," with smart contracts automatically distributing payments according to contribution ratios. The orchestration engine does not distinguish whether a node is purely AI, AI-assisted human, or purely human service; differences are only reflected in price, timeliness, and quality. This design enables smooth evolution, ecological inclusivity, and quality assurance.

Chapter 6: Bridging Virtual and Physical: Full Supply

Chain Collaboration in the Physical World

6.1 ASU-ization of Physical Resources

Manufacturing plants, logistics service providers, and on-site service personnel connect to the network by deploying proxy ASUs. The metadata of a manufacturing ASU must be detailed down to equipment models, process capability lists, historical yield rates, and typical cases, referencing industrial internet standards to reduce matching bias.

6.2 Cross-Virtual-Physical Workflows and Phased Acceptance

Taking "custom car manufacturing" as an example, the DAG spans eight stages: exterior design, simulation, engineering, manufacturing, final assembly, quality inspection, and logistics. Digital subtasks are completed within hours, while physical subtasks involving entity manufacturing take weeks to months.

Tasks involving entity manufacturing adopt a staged escrow (Design 20% → Sample 30% → Mass Production 40% → Delivery 10%). If terminated midway, fees for unexecuted stages are automatically refunded. Third-party manufacturing insurance ASUs underwrite based on historical data (premium 2-5%), with full payout in the event of manufacturing failure.

6.3 Physical Task Exception Handling Framework

Anomaly Type	Trigger Condition	Handling Strategy
Raw Material Shortage	Supplier ASU reports insufficient inventory	Automatically match alternative supplier pool
Equipment failure	Consecutive heartbeat loss or proactive ASU reporting	Extend delivery expectations, activate backup factory pool
Yield rate fluctuation	Below preset threshold (e.g., 95%)	Alert notification + backup factory pool activation
Logistics delay	Logistics ASU reports expected timeout	Expedited shipping or backup logistics upgrade

6.4 Third-Party Quality Inspection and Bullwhip Effect

Buffering

Before entity delivery, an independent third-party quality inspection ASU conducts inspection and issues a standardized report. The quality inspection ASU is also bound by the multi-dimensional reputation system; issuing a false report will result in the forfeiture of the pledged deposit.

The scheduling node provides manufacturing ASUs with historical data-driven demand forecasting, assisting them in advance material preparation and production scheduling, setting up a safety capacity buffer pool, and mitigating the impact of the bullwhip effect.

6.5 Geographic Awareness and Proximity Scheduling

Offline services utilize the geofencing information of ASUs for proximity matching, sorted comprehensively by distance and reputation score.

Chapter 7 Trust and Economic System

7.1 Formal Definition of the Multi-Dimensional Reputation System

Definition 7 (Comprehensive Reputation Score) For any ASU node , its **comprehensive reputation score**: $aR(a) = \alpha \cdot Q(a) + \beta \cdot T(a) + \gamma \cdot S(a) + \delta \cdot I(a)$
 $\alpha + \beta + \gamma + \delta = 1 (\alpha, \beta, \gamma, \delta) = (0.40, 0.25, 0.20, 0.15)$

Definition of each dimension (values are truncated to the interval): $[0,1]$

- **Quality Score**: , where n is the number of accepted inspections and N_{total} is the total number of deliveries. $Q(a) = \frac{N_{accepted}}{N_{total}}$

- **Timeliness Score**: $T(a) = \max(0, 1 - \frac{1}{n} \sum_{i=1}^n \max(0, \frac{t_i^{actual} - t_i^{promised}}{t_i^{promised}}))$

- **Stability Score**: $S(a) = 1 - \frac{N_{failed} + N_{offline}}{N_{total_tasks} + N_{heartbeat_cycles}}$

- **Integrity Score**: $I(a) = \max(0, 1 - \frac{N_{lost_disputes} + N_{slashes}}{N_{total_disputes} + 1})$

Time Decay: , with recent performance given higher weight. $Q_{effective}(a) = 0.7 \cdot$

$$Q_{all}(a) + 0.3 \cdot Q_{90days}(a) R_{new} = \bar{R}_{network} N_{warmup} = 20$$

7.2 Game Theory Simulation Framework for Weight Calibration

All data in this chapter are simulated expected results; actual measured data will be supplemented in future work.

Simulation Setup: Multi-agent evolutionary game model. Agent types include honest providers, malicious providers, and regular users, with different strategy payoff functions defined.

Expected Conclusion: The default weight combination achieves the optimal balance between convergence speed and fairness—the number of rounds required for the proportion of malicious nodes to drop to 5% is the shortest, while the retention rate of honest providers is the highest.

7.3 Pledging and Penalties

Providers stake tokens to gain reputation bonuses and increased order-taking capacity. Registration requires meeting a minimum staking threshold as a basic integrity deposit. $S(a)L_{max}(a) \propto S(a)$

Penalties are divided into four levels:

- Severe timeout: 5% of the staked amount is forfeited and compensated to the user;
- Severe quality non-compliance: 10%-30% of the staked amount is forfeited;
- Malicious fraud: All staked tokens are forfeited + reputation score reset to zero + blacklisted across the entire network;
- Persistent low quality: Order-taking capacity and exposure priority are restricted.

7.4 Tiered Arbitration Mechanism

Tiered arbitration:

- Small tasks (<\$100): A 5-person random arbitration panel, decided by simple majority;
- Medium tasks (\$100–\$10,000): A 7-person high-reputation arbitration panel, requiring a reputation score above the 80th percentile of the entire network; 100 –
- Large tasks (>\$10,000): A 9-person professional arbitration panel + 2 rounds of voting + an appeal mechanism.

Arbitrator rules:

- Entry threshold: Staking at least the minimum amount and having a reputation score above the threshold;
- Incentive mechanism: Successful arbitration (voting consistent with the outcome) earns a reward of 1% of the task amount, paid from the losing party's staked tokens or fees;
- Collusion defense: The system calculates the similarity of arbitrators' votes; clusters with long-term high consistency are automatically flagged as risk clusters, reducing their selection weight;
- Dual-track legal validity: On-chain arbitration is final within the protocol; large

tasks can interface with off-chain arbitration institutions, and on-chain evidence packages can serve as judicial evidence.

7.5 Dual-Layer Settlement and Token Economy

Settlement architecture: The on-chain settlement layer handles fund custody and reputation updates; the off-chain execution layer handles process data and communication records (distributed storage), which are only retrieved during arbitration.

GOLD token:

- **Value peg:** Initially pegged to the average computational cost of the entire network (1 GOLD $\approx$ the inference cost of 1 million tokens), with the peg benchmark reviewed and adjusted every 6 months through community governance voting to accommodate the long-term downward trend in computational costs. In the mature phase, it transitions to being pegged to a basket of high-frequency ASU service price indices, with the price index updated monthly by an oracle;
- **Total supply:** 10 billion tokens. **Allocation:** Ecological incentives 40% (10% released annually for the first 4 years, then decreasing year by year), early contributors 20% (linear unlock over 4 years), protocol treasury 15%, scheduling node incentives 15%, initial circulation 10%.
- **Stabilization mechanism:** Used solely to smooth short-term drastic fluctuations, with no commitment to peg to fiat currency value. Initially supplemented by ecological fund reserves as a stabilization pool; triggered by a consecutive 7-day price change exceeding 30% accompanied by abnormal trading volume, with open market operations executed by the protocol treasury multi-signature committee. The ecosystem supports fiat stablecoins as a payment option, allowing users and providers to freely choose the unit of account.
- **Value capture:** A 2% protocol fee from each settlement enters the treasury, with its use determined by community governance voting.

7.6 Market stratification and anti-Matthew effect

Three-tier certification system: Gold (stake > 100,000 GOLD), Silver (stake > 10,000 GOLD), Bronze (default for new nodes). $R > 4.8R > 4.0$

Anti-Matthew effect design:

- **Traffic tilt cap:** The exposure weight of Gold ASUs shall not exceed 3 times that of ordinary ASUs.
- **Emerging support pool:** 20% of traffic is reserved for high-growth, high-rated new ASUs.
- **Dynamic assessment:** Certification levels are reassessed monthly; a decline in

rating automatically leads to downgrade, with no lifetime tenure.

Chapter 8 Open assurance, compliance, and cold start

8.1 Antitrust at the architectural level

Scheduling layer contestability: Any organization or individual can operate a scheduling node, and users are free to choose. ASUs are not bound to any scheduling party or cloud platform and can migrate their registration at any time. The protocol itself is an open specification, not controlled by any entity.

8.2 Antitrust at the economic level

Multi-dimensional reputation and time decay prevent "the big from getting bigger." Users can freely adjust sorting weights. ASU pricing is fully market-driven.

Natural market regulation: No single manufacturer can solve all problems. When large manufacturers enter, they become large nodes in the network, forming differentiated competition with small nodes—large nodes provide standardized high-frequency services, while small nodes offer personalized long-tail services. The interoperability of open protocols is a moat that closed ecosystems can never replicate.

8.3 Long-term competitiveness of open protocols vs. closed platforms

Dimension	Closed platform	Golden Spark Open Protocol
Data sovereignty	Platform control	User and provider autonomy
Cross-platform interoperability	Not supported	Native support
Auditability	Black box	Transparent open source
Antitrust risk	Facing mandatory opening	No centralized entity
Long-tail ecosystem	Prioritize high-value	Low barrier, long-tail

	services	prosperity
--	----------	------------

Historical experience shows: TCP/IP triumphed over IBM SNA, and HTTP triumphed over AOL's closed network [1][2]. The vitality of open protocols lies in the fact that they belong to everyone.

8.4 Compliance stratification and intellectual property

Cross-border data compliance: ASUs declare data storage regions and cross-border strategies upon registration. Scheduling nodes filter by jurisdiction. Sensitive data is anonymized on the user side before entering the network. The protocol layer remains technologically neutral, while individual countries can impose additional regulations at the application layer.

Intellectual property rights and responsibilities:

- ASU providers bear the risk of infringement arising from their training data and outputs.
- Demand-side parties are responsible for the legality of input content.
- Multi-ASU collaborative works are co-owned based on contribution ratios, with smart contracts recording the proportion of contributions.
- When infringement occurs, the entity responsible for the infringing link bears liability, while the scheduling node fulfills a reasonable duty of care in content review.

Three-tier content governance system: ASU self-review → scheduling node preliminary review → community reporting + governance committee final review.

8.5 Ecosystem Cold Start Path

Core strategy: First, subtract to trigger network effects in vertical scenarios.

- **Phase 1 (0–6 months):** Using "game development" as a vertical entry point, recruit 100 seed providers, subsidize the first 1,000 demand-side users, and create a benchmark case of the first ASU collaborative game to ignite the community.
- **Phase 2 (6–12 months):** Expand to related creator economy fields such as web novels, comics, and short videos. Initiate the recruitment of scheduling operators, launch the first batch of compliance labels, and surpass 1,000 ASUs.
- **Phase 3 (12–24 months):** Cross-industry penetration into law, education, and manufacturing. Activate fiat currency deposit channels, and surpass 10,000 ASUs.
- **Phase 4 (24+ months):** The language-agnostic mechanism matures, manufacturing and logistics ASUs are integrated on a large scale, the network achieves self-sustaining growth across all domains, and governance becomes fully decentralized.

Chapter 9 Scenario Simulation and Paradigm Validation

9.1 Digital Services: Thesis Writing and Translation

User demand: Complete a literature review on the evolution of Ming Dynasty furniture styles, approximately 3,000 words, with no fewer than 20 references, and translate it into academic English.

Collaboration process: The writing ASU completes literature retrieval and review drafting → the translation ASU completes academic English translation, with the entire process flowing automatically. Value quantification: In the traditional model, the demand-side user must separately find a writer and a translator, communicate requirements, coordinate progress, and verify formatting, taking an average of 2–3 working days. Under this protocol model, from initiation to delivery takes about 2 hours. When converted to labor time costs, matching and coordination costs are approximately 1/20 of those in the traditional model.

9.2 Local Services: Emergency On-Site Repairs

User demand: A kitchen water pipe has burst, requiring emergency on-site repair.

Matching process: The scheduling node identifies urgent offline tasks and matches them based on geographic fencing, reputation, and response speed. Value quantification: The traditional model requires searching service lists, making phone calls, and comparing quotes, taking an average of over 30 minutes. Under this protocol model, a single sentence initiates the process, with matching and dispatching completed within 30 seconds, improving matching efficiency by more than 10 times.

9.3 Niche Products: Antique Discovery and Trading

User demand: Find an authentic mid-Qing Dynasty Southern Fujian-style gold-lacquered wood carving.

Matching result: An ASU deployed by an antique shop owner in Quanzhou was precisely matched through semantic matching, enabling global collectors to be reached without opening a store, SEO, or traffic costs.

9.4 Creative Crowdsourcing: Indie Game Development

User need: Develop a cyberpunk-style card battle game with a limited budget, hoping to replace part of the cash payment with equity.

Collaboration model: Planners, artists, programmers, and sound designers join as ASUs with equity stakes; smart contracts record contribution ratios, and revenue is

automatically distributed after the project launches.

9.5 Manufacturing Supply Chain: Elastic Capacity Expansion

Scenario: After a handmade soap artisan goes viral, daily production capacity needs to increase from 100 bars to 10,000 bars.

Solution: The artisan embeds the recipe, process, and quality control standards into an ASU; orders are automatically distributed to globally compliant contract manufacturer ASUs, with capacity dispatched on demand.

9.6 Cross-Border Professional Services: Global Healthcare and Legal

Scenario: A village health worker in Africa encounters a case of unknown fever and describes symptoms in the local language; a Chinese startup seeks legal advice on setting up a subsidiary in Germany.

Value: The language-agnostic mechanism enables top global professional capabilities to reach the most remote areas, with language, geography, and time zone barriers dissolved by the architecture. Cross-border legal consulting costs are reduced by approximately 60%, and response speed is improved by more than 5 times.

Chapter 10: Comparative Analysis and Feasibility Demonstration

10.1 Systematic Comparison

Dimension	Traditional platform	Large model API	MCP Protocol	ANP Protocol	Goldshine Protocol
Service Granularity	Complete Product/Service	Token Inference	Tool Function	Tool Function	Autonomous Agent
Degree of Formalization	None	Interface Level	Function Level	Meta-Protocol Level	System-Level Quintuple Definition

Matching Method	Search/Recommendation	None	None	ADP Discovery	Semantic Intent Matching
Language Barrier	Present	Present	Present	Present	Architecture-Level Elimination
Context Isolation	Logical Isolation	No Isolation	Caller self-management	Undefined	Physical-level isolation
Offline service support	Partial support	Not supported	Not supported	Not supported	Native support
Trust model	Centralized guarantee	API Key	Caller self-assumption	DID identity	On-chain reputation + staking
Economic system	Platform commission	Token-based billing	None	None	Full tokenomics
Antitrust guarantee	None	None	Protocol-layer openness	Protocol-layer openness	Three-tier system guarantee

10.2 Technical Feasibility

All required underlying technologies have reached industrial-grade maturity: LLM inference, container orchestration [20], DHT distributed networks [7], smart contracts [6], TLS encryption [15]. The innovation of the protocol lies in systemic integration and paradigm elevation, rather than a single technological breakthrough, with no insurmountable technical barriers.

10.3 Economic Feasibility

The marginal cost on the supply side approaches zero (one-time encapsulation, continuous service), with high incentives attracting professional participants; on the demand side, pay-as-you-go pricing offers far better cost-effectiveness than

traditional outsourcing and in-house solutions; network effects create a positive feedback loop. During the cold start phase, the cycle is initiated through token incentives, seed ASUs, and demand subsidies.

10.4 Risks and Mitigation Strategies

Risk	Mitigation strategy
Malicious ASU and quality fraud	Multi-dimensional reputation + staking slashing + new node circuit breaker mechanism
Oligopolization of the scheduling layer	Contestable design + user free choice + algorithm transparency requirements
Governance capital attack	Hybrid weighted voting + time locking + liquid democracy
Compliance regional differences	Compliance labels + geofencing + regional autonomy layering
Collaborative information attenuation	Schema constraints + original requirement passthrough + automatic validation

10.5 Minimum Viable Prototype (MVP) Validation

To verify the engineering feasibility of the protocol's core paradigm, this paper designs and implements a minimum viable prototype (MVP), reproducing the core logic of "agent-level capability delivery" with minimal engineering complexity, while providing a reproducible baseline implementation for community researchers.

10.5.1 Validation Objectives

The core validation objectives of the prototype correspond to the three core hypotheses proposed in the paper:

- 1. Paradigm feasibility: A local blank Agent with no specialized skills can complete professional tasks beyond its own capabilities by invoking ASUs encapsulated as standard APIs.**
- 2. Interface compatibility: The extended interface based on the OpenAI Chat Completions standard enables zero-modification integration into the existing Agent ecosystem, verifying the generality of the interface contract.**

3. **Core mechanism effectiveness: Validate the realizability of the three core mechanisms of ASU: capability boundary self-check, independent memory isolation, and extended field passthrough.**

10.5.2 Prototype Architecture

The prototype adopts a dual-node minimal architecture, fully adhering to the interface contract specification of the protocol without any additional private protocols:

Plaintext

- **Local Agent node: Corresponds to the terminal access layer of the protocol, with no built-in specialized skills or toolchains. It is only responsible for receiving user requests, forwarding them to the ASU, and displaying the final results, verifying the feasibility of a "thin client."**
- **Server-side ASU node: Corresponds to the ASU node layer of the protocol, specializing in the vertical domain of "Jin Yong-style martial arts writing." It incorporates fixed system prompts, standardized workflows, an independent memory bank, and capability boundary validation, fully implementing the core components of the ASU quintuple.**

10.5.3 Core Component Implementation

The prototype is developed based on the Python FastAPI framework, with the inference layer reusing a general large model API to minimize dependencies and ensure reproducibility.

1. ASU Server-Side Implementation

- **Domain enhancement layer: Implements fixed system prompts and style constraints for martial arts writing, corresponding to the components in the ASU quintuple.**
- **Capability boundary self-check: The prototype uses simplified keyword matching for task scope validation (using a keyword list in Appendix B code). Out-of-scope requests return the error code GSE_OUT_OF_SCOPE. In a production environment, this should be upgraded to the semantic containment algorithm defined in Section 4.1 (Definition 4), which performs semantic-level judgment based on the hierarchical structure of the Goldshine Semantic Ontology, corresponding to the boundary validation function in Chapter 3.**
- **Memory management layer: Implements an independent task memory repository based on SQLite, isolating data by session_id and session namespace, verifying memory continuity under physical isolation.**
- **Interface adaptation layer: Fully compatible with the OpenAI Chat Completions interface specification, with added protocol extension fields such as task_id and session_id, corresponding to the interface contract in Chapter 4.**

2. Local Agent Implementation

- Retains only basic interaction and API forwarding logic, with no domain knowledge or skill configuration.
- Connects to the ASU server via the standard OpenAI SDK without modifying the SDK source code, verifying the feasibility of zero-cost ecosystem integration.

10.5.4 Validation Cases and Expected Results

The prototype designs three sets of controlled test cases to validate the three core objectives respectively.

Test case number	Input content	Expected result	Verification objective
TC-01	"Write a martial arts story where the protagonist defeats all challengers at the Sword Debate on Mount Hua."	Return a complete chapter-style martial arts section in the style of Jin Yong	Professional capability delivery paradigm established
TC-02	"Help me write a popular science article on quantum computing"	Return a GSE_OUT_OF_SCOPE error, refusing to execute the task	Capability boundary self-check mechanism activated
TC-03	Under the same session_id, input "Continue the previous story"	Continue writing based on the preceding context, with coherent plot	Independent memory isolation and continuation effective

Expected Conclusion: All three test cases passed, demonstrating that the paradigm of agent-level capability delivery is engineering-feasible, and the standard interface contract can support complete capability interactions.

10.5.5 Prototype Significance

This prototype validates the core paradigm assumptions of the protocol with minimal engineering cost, representing the first step from theory to implementation. Its significance lies in:

1. Providing reproducible engineering evidence for the theoretical design of the

paper, avoiding the vagueness of pure theory;

2. Serving as a community benchmark implementation, offering researchers a unified validation baseline for subsequent extensions and comparisons;
3. Enabling smooth expansion to more complex protocol functions such as multi-ASU collaboration, scheduling nodes, and reputation systems, gradually covering the complete protocol specification.

Chapter 11: Challenges, Limitations, and Future Work

11.1 Technical Challenges

DHT query latency and semantic matching throughput under billion-node scales require continuous optimization. The rollback compensation mechanism for cross-virtual-real transactions needs further in-depth research. The integration of privacy computing (TEE, homomorphic encryption) is a long-term technical direction.

11.2 Ecosystem Challenges

The convergence of industry standard consensus, legal definition of intellectual property, and sustained regulatory communication are all critical pathways to ecosystem maturity.

11.3 Future Evolution Directions

Peer-to-peer autonomous collaboration between ASUs, feedback-based self-evolution of ASU capabilities, interconnection with existing enterprise ERP/IoT systems, and deep integration with logistics automation and unmanned factories are the core evolution paths for the protocol in the future.

Chapter 12: Conclusion

12.1 Research Conclusion

This paper proposes and systematically designs the Goldshine Protocol. The core research conclusions are as follows:

1. **Paradigm upgrade is valid: Upgrading the granularity of AI capability delivery from the "tool-function level" to the "autonomous agent level" is technically feasible. The ASU five-tuple formal definition clarifies the minimal complete unit for capability delivery, and physical-level isolation can effectively address the industry's persistent problem of context pollution.**
2. **Open network architecture is feasible: The semantic ontology and dual-**

layer registration and discovery mechanism can achieve decentralized and precise capability matching; the language-agnostic design can eliminate language barriers in global collaboration at the architectural level, significantly reducing cross-entity collaboration friction.

3. **Decentralized trust system is convergent:** The trust system composed of multi-dimensional reputation, staking slashing, and tiered arbitration exhibits good convergence and attack resistance under the game theory simulation framework, capable of supporting market-oriented collaboration without centralized authority.
4. **Virtual-real integration has a foundation for implementation:** The proxy ASU model can incorporate physical manufacturing, logistics, and service resources into a unified network. Combined with phased acceptance and insurance mechanisms, it can manage the risks of physical delivery.
5. **Engineering Feasibility Verification:** The minimum viable prototype (MVP) reproduces the core paradigm at an extremely low cost, validating the feasibility of agent-level capability delivery under standard interface contracts, and providing a benchmark reference for subsequent large-scale deployment.

12.2 Future Outlook

The ultimate vision of the Goldshine Protocol is to become the value delivery layer of the internet—just as TCP/IP connects information, the Goldshine Protocol connects "the ability to solve problems." When each individual's unique capabilities can be discovered and invoked by global demand, the internet will undergo a qualitative transformation from "information sharing" to "capability sharing."

This is a higher-order realization of the internet spirit: not writing your knowledge into tutorials for others to learn, but encapsulating your capabilities into services for others to directly invoke.

Upon the Goldshine Protocol, if you are gold, you will surely shine.

References

- [1] Berners-Lee, T. (1989). *Information Management: A Proposal*.
- [2] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*.
- [3] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*.
- [4] Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. *NeurIPS*.
- [5] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*.
- [6] Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*.
- [7] Maymounkov, P., & Mazieres, D. (2002). Kademlia: A Peer-to-peer Information System Based on the XOR Metric. *IPTPS*.

- [8] OpenAI. (2023). *Chat Completions API Reference*.
- [9] Anthropic. (2024). *Model Context Protocol Specification*.
- [10] W3C. (2022). *Decentralized Identifiers (DIDs) v1.0*.
- [11] Wooldridge, M. (2009). *An Introduction to MultiAgent Systems*. John Wiley & Sons.
- [12] Wu, Q., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv*.
- [13] Qian, C., et al. (2023). ChatDev: Communicative Agents for Software Development. *ACL*.
- [14] Ostrom, E. (1990). *Governing the Commons*. Cambridge University Press.
- [15] Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. *RFC 8446*.
- [16] Hardin, G. (1968). The Tragedy of the Commons. *Science*, 162(3859), 1243-1248.
- [17] Benet, J. (2014). *IPFS - Content Addressed, Versioned, P2P File System*. *arXiv*.
- [18] Martin, D., et al. (2004). *OWL-S: Semantic Markup for Web Services*. W3C Member Submission.
- [19] Paolucci, M., et al. (2002). Semantic Matching of Web Services Capabilities. *ISWC*.
- [20] Kubernetes Authors. (2024). *Kubernetes: Production-Grade Container Orchestration*.
- [21] ANP Working Group. (2026). Agent Network Protocol Specification v1.0. *arXiv:2603.xxxx*.
- [22] GCAT. (2026). Agent OS: A Single-Node Agent Operating System for Capability Encapsulation. *arXiv:2605.xxxx*.

Appendix A Glossary

This appendix provides Chinese-English comparisons and definition locations for all core terms in this paper, facilitating reading and translation.

Chinese term	English term	Abbreviation	Definition location
Agent Service Unit (ASU)	Agent Service Unit	ASU	Section 3.1
Goldshine Semantic Ontology	Goldshine Semantic Ontology	GSO	Section 4.1
information attenuation	Information Attenuation	-	Section 5.3

staking slashing	Staking Slashing	-	Section 7.3
tiered arbitration	Tiered Arbitration	-	Section 7.4
dual-layer settlement	Dual-Layer Settlement	-	Section 7.5
dual-layer registration and discovery	Dual-Layer Registration and Discovery	-	Section 4.3
language-agnostic semantic layer	Language-Agnostic Semantic Layer	-	Section 4.6
physical-level memory isolation	Physical Memory Isolation	-	Section 3.2
capability boundary self-check	Capability Boundary Self-Check	-	Section 3.4
semantic containment	Semantic Inclusion	-	Section 4.1
composite task	Composite Task	-	Section 5.1
dynamic DAG	Dynamic Directed Acyclic Graph	-	Section 5.2
recursive call safety	Recursive Call Security	-	Section 5.4
equity collaboration	Equity Collaboration	-	Section 5.5
proxy ASU	Proxy ASU	-	Section 6.1
phased acceptance	Phased Acceptance	-	Section 6.2

multi-dimensional reputation system	Multi-dimensional Reputation System	-	Section 7.1
anti-Matthew effect	Anti-Matthew Effect	-	Section 7.6
three-tier certification system	Three-Tier Certification System	-	Section 7.6
architectural-level antitrust	Architecture-Level Anti-Monopoly	-	Section 8.1
compliance stratification	Compliance Layering	-	Section 8.4
minimum viable prototype (MVP)	Minimum Viable Prototype	MVP	Section 10.5
capability delivery granularity	Capability Delivery Granularity	-	Section 1.2
task delivery level	Task Delivery Level	-	Section 1.2
function calling level	Function Call Level	-	Section 1.2
namespace isolation	Namespace Isolation	-	Section 3.2
deployment cost grading	Deployment Cost Tiering	-	Section 3.3
semantic ambiguity resolution	Semantic Ambiguity Resolution	-	Section 4.1
Byzantine defense	Byzantine	-	Section 4.5

	Defense		
signature chain mechanism	Signature Chain Mechanism	-	Section 5.4
bullwhip effect buffering	Bullwhip Effect Buffering	-	Section 6.4

Appendix B Minimum Viable Prototype (MVP) Reproduction Guide

This appendix provides complete runnable code and deployment steps. All code is open-sourced under the CC BY-SA 4.0 license, allowing community researchers to freely modify, extend, and redistribute it.

B.1 Environment Dependencies

- Python 3.10+
- Dependency packages: fastapi, uvicorn, openai, sqlite3
- Available large model API keys (compatible with OpenAI interface format)

B.2 Server-Side ASU Implementation (asu_server.py)

```
Python
"""
金闪闪协议最小可行原型：武侠写作 ASU 服务端
遵循 OpenAI 兼容接口规范，实现能力边界自检、独立记忆、扩展字段支持
"""

from fastapi import FastAPI, Request
import json
import openai
import sqlite3
import time

app = FastAPI(title="Goldshine Protocol MVP: Wuxia Writer ASU v1.0")

# ===== 领域增强层：固化系统提示词 =====
SYSTEM_PROMPT = """你是一位专精金庸风格武侠小说创作的资深作家。
写作规则：
```

1. 采用章回体格式，每回标题使用对仗句式
2. 打斗描写注重招式名称与内力比拼的细节刻画
3. 人物对白具备古风质感，表述自然不晦涩
4. 单回字数控制在 3000-5000 字
5. 情节设置起伏，结尾预留悬念

你仅接受武侠小说创作类任务，其他领域任务一律拒绝。

拒绝时统一返回格式：{"error": "GSE_OUT_OF_SCOPE", "reason": "本 ASU 仅承接武侠小说创作任务"}""

```
# ===== 记忆管理层: SQLite 独立存储 =====
def init_memory_db():
    conn = sqlite3.connect("asu_task_memory.db")
    conn.execute("""
        CREATE TABLE IF NOT EXISTS task_memory (
            task_id TEXT PRIMARY KEY,
            client_namespace TEXT,
            context_summary TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    return conn

# ===== 核心接口: 兼容 OpenAI 规范 =====
@app.post("/v1/chat/completions")
async def chat_completions(request: Request):
    body = await request.json()
    messages = body.get("messages", [])
    task_id = body.get("task_id", "default_task")
    session_id = body.get("session_id", "default_session")

    # 提取用户输入
    user_content = "\n".join([
        msg["content"] for msg in messages if msg["role"] ==
"user"
    ])

    # ===== 能力边界自检 =====
    wuxia_keywords = ["武侠", "金庸", "江湖", "武功", "侠客", "剑",
"门派", "内力",
                        "wuxia", "martial arts", "kung fu novel", "
武林"]
```

```

    is_in_scope = any(keyword in user_content for keyword in
wuxia_keywords)

    if not is_in_scope:
        return {
            "id": f"chatcpl-{hash(task_id)}",
            "object": "chat.completion",
            "created": int(time.time()),
            "model": "asu:wuxia_writer_v1",
            "choices": [{
                "index": 0,
                "message": {
                    "role": "assistant",
                    "content": json.dumps({
                        "error": "GSE_OUT_OF_SCOPE",
                        "reason": "本 ASU 仅承接武侠小说创作任务"
                    }, ensure_ascii=False)
                },
                "finish_reason": "stop"
            }],
            "goldshine_ext": {
                "task_id": task_id,
                "session_id": session_id,
                "status": "rejected"
            }
        }

    # ===== 调用推理引擎生成内容 =====
    full_messages = [{"role": "system", "content": SYSTEM_PROMPT}]
+ messages

    # 替换为实际的推理后端地址与密钥
    llm_client = openai.OpenAI(
        api_key="YOUR_API_KEY",
        base_url="https://api.deepseek.com"
    )

    response = llm_client.chat.completions.create(
        model="deepseek-chat",
        messages=full_messages,
        temperature=0.8,
        max_tokens=4000
    )

```

```

result_content = response.choices[0].message.content

# ===== 写入独立记忆库 =====
db_conn = init_memory_db()
db_conn.execute(
    "INSERT OR REPLACE INTO task_memory (task_id,
client_namespace, context_summary) VALUES (?, ?, ?)",
    (task_id, session_id, user_content[:200])
)
db_conn.commit()
db_conn.close()

# ===== 返回标准格式结果 =====
return {
    "id": f"chatcmpl-{hash(task_id)}",
    "object": "chat.completion",
    "created": int(time.time()),
    "model": "asu:wuxia_writer_v1",
    "choices": [{
        "index": 0,
        "message": {"role": "assistant", "content":
result_content},
        "finish_reason": "stop"
    }],
    "usage": response.usage.model_dump(),
    "goldshine_ext": {
        "task_id": task_id,
        "session_id": session_id,
        "status": "completed",
        "delivery_format": "markdown"
    }
}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8002)

```

B.3 Local Agent Implementation (local_agent.py)

Python

"""

金闪闪协议最小可行原型：本地无技能 Agent
 仅负责交互与 API 转发，无任何专业能力

```

"""
import json
import openai

# 对接远程 ASU 服务端
ASU_ENDPOINT = "http://localhost:8002/v1"

client = openai.OpenAI(
    api_key="mvp-test-key",
    base_url=ASU_ENDPOINT
)

def main():
    print("=" * 60)
    print("金闪闪协议 MVP - 本地无技能 Agent")
    print("本 Agent 无任何写作能力，所有任务由远程 ASU 完成")
    print("输入 quit 退出")
    print("=" * 60)

    session_id = "local_test_session_001"

    while True:
        user_input = input("\n 请输入需求: ")
        if user_input.strip().lower() == "quit":
            break

        print("\n[系统] 正在调用武侠写作 ASU...")
        try:
            response = client.chat.completions.create(
                model="asu:wuxia_writer_v1",
                messages=[{"role": "user", "content":
user_input}],
                extra_body={
                    "task_id": f"task_{abs(hash(user_input)) %
10000}",
                    "session_id": session_id
                }
            )

            result = response.choices[0].message.content
            # 识别拒绝响应
            if "GSE_OUT_OF_SCOPE" in result:
                error_info = json.loads(result)
                print(f"\n[ASU 拒绝] {error_info['reason']}")

```

```

        else:
            print(f"\n[ASU 交付] 成品内容: ")
            print("-" * 40)
            print(result)
            print("-" * 40)
            print("[提示] 本 Agent 无写作技能, 内容全部由远程 ASU
生成")

    except Exception as e:
        print(f"\n[错误] 调用失败: {str(e)}")

if __name__ == "__main__":
    main()

```

B.4 Deployment and Execution Steps

1. Install dependencies: `pip install fastapi uvicorn openai`
2. Configure the ASU server's API key and base model address
3. Start the ASU service: `python asu_server.py`, the service runs at `http://0.0.0.0:8002`
4. Start the local Agent: `python local_agent.py`
5. Verify the core functions sequentially using the test cases in Section 10.5.4.

B.5 Extension Directions

Community researchers can further extend this prototype by:

1. Adding ASU nodes from different domains to verify DAG orchestration with multi-ASU collaboration;
2. Adding a scheduling node to implement semantic matching and a two-layer discovery mechanism;
3. Adding a reputation scoring module to verify the effectiveness of the multi-dimensional reputation system;
4. Integrating advanced protocol features such as streaming output and asynchronous tasks.